Experiment 6

NAME: Vedant Naik
ROLLNO: 41
CLASS: SE-2

Title

Create a webpage that retrieves, displays, and updates user data from a MySQL database. Implement CRUD operations (Create, Read, Update, Delete) using PHP & MySQL. Ensure user input is validated and securely stored in the database. Add Session & Cookies.

Theory:

Modern web applications often require interaction with databases to store, retrieve, and manage user information efficiently. This experiment focuses on developing a dynamic web application using PHP and MySQL that performs CRUD operations (Create, Read, Update, Delete) on user data. Additionally, the application ensures input validation, secure data storage, and incorporates Sessions and Cookies for user management

A. PHP and MySQL Integration

PHP (Hypertext Preprocessor) is a server-side scripting language widely used for web development. It is especially suitable for database-driven applications because it can easily interact with databases like **MySQL**.

MySQL is an open-source relational database management system (RDBMS) used to store structured data in tables. PHP communicates with MySQL using extensions such as **MySQLi** or **PDO (PHP Data Objects)** to execute SQL queries.

In this experiment, PHP scripts handle database connections, execute SQL commands, and dynamically generate HTML output based on database records.

B. CRUD Operations

CRUD represents the four fundamental database operations:

1. Create (Insert)

The **Create** operation is used to add new user records into the database. User data is collected through HTML forms and sent to the server using HTTP POST method. PHP processes the input and inserts validated data into the MySQL table using the INSERT SQL statement.

2. Read (Select)

The **Read** operation retrieves user data from the database and displays it on the webpage. PHP executes the SELECT SQL query and fetches records, which are displayed dynamically in tabular format. This operation allows users to view stored data.

3. Update

The **Update** operation modifies existing user records. The selected record is loaded into an HTML form, allowing the user to edit data. PHP then updates the record in the database using the UPDATE SQL statement.

4. Delete

The **Delete** operation removes unwanted records from the database using the DELETE SQL statement. This operation helps maintain accurate and relevant data.

C. Database Structure

A MySQL database typically contains:

- A **database** to store application data
- A **table** to store user information such as ID, name, email, and password
- A **primary key** to uniquely identify each record

Relational databases ensure data consistency and integrity through structured schema and constraints.

D. Input Validation

Input validation is essential to ensure that user-provided data is correct and safe. Validation includes:

- Checking for empty fields

- Validating email format
- Restricting input length
- Allowing only valid characters

Validation can be implemented using both **client-side validation (HTML/JavaScript)** and **serverside validation (PHP)**. Server-side validation is critical, as client-side validation can be bypassed.

E. Security Measures

SQL Injection Prevention

SQL Injection is a common attack where malicious SQL code is inserted into input fields. To prevent this:

- Prepared statements
- Parameterized queries are used to securely execute SQL commands.

Password Security

Passwords should never be stored in plain text. PHP provides hashing functions such as password_hash() to securely store passwords and password_verify() to authenticate users.

F. Sessions

A **session** is a server-side mechanism used to store user data temporarily across multiple pages. Sessions are commonly used for:

- User login systems
- Authentication
- Maintaining user state

In PHP, sessions are created using session_start() and data is stored in session variables. Session data is automatically destroyed when the browser is closed or the session expires.

G. Cookies

A **cookie** is a small piece of data stored on the client's browser. Cookies are used to:

- Remember user preferences
- Store login information
- Track user activity

Unlike sessions, cookies have an expiration time and remain stored even after the browser is closed. PHP uses the setcookie() function to create cookies.

H. Dynamic Web Page Behavior

The application dynamically updates content based on database operations. PHP fetches updated records after every CRUD operation, ensuring that the displayed data always reflects the current state of the database.

Algorithm/Procedure:

A. Database Creation

1. Start XAMPP/WAMP server.
2. Open phpMyAdmin.
3. Create a database named userdb.

B. CRUD Operations Flow

4. Create a table users with fields:

○ id (INT, Primary Key, Auto Increment) ○ name

(VARCHAR) ○ email (VARCHAR) ○ password

(VARCHAR)

Steps:

1. User enters details in HTML form.
2. PHP validates user input.
3. Data is securely inserted into MySQL database.
4. Existing records are fetched and displayed.
5. User can update or delete records.
6. Sessions manage login state.
7. Cookies store user preference.

Program/Code:

1. Database Connection (db.php)

```
<?php
$conn = mysqli_connect("localhost", "root", "", "userdb");
if (!$conn) { die("Database connection failed");
}
?>
```

2. Create (Insert) User

```
<?php
include "db.php"; if
(isset($_POST['submit'])) {
```

```
    $name = trim($_POST['name']);

    $email = trim($_POST['email']);
    $pass = password_hash($_POST['password'], PASSWORD_DEFAULT); if
    (!empty($name) && !empty($email)) {
        $query = "INSERT INTO users(name,email,password) VALUES('$name','$email','$pass')";
        mysqli_query($conn, $query); header("Location: index.php");
    }
}
?>
```

3. Read (Display Data) – index.php

```
<?php include "db.php"; ?>
<html>
<body>
<h2>User List</h2>

<table border="1">
<tr>
<th>ID</th><th>Name</th><th>Email</th><th>Action</th>
</tr>

<?php
```

```
$result = mysqli_query($conn, "SELECT * FROM users");
while ($row = mysqli_fetch_assoc($result)) {

?>
<tr>
<td><?= $row['id']; ?></td>
<td><?= $row['name']; ?></td>
<td><?= $row['email']; ?></td>
<td>
<a href="edit.php?id=<?= $row['id']; ?>">Edit</a> |
<a href="delete.php?id=<?= $row['id']; ?>">Delete</a>
</td>
</tr>
<?php } ?>
</table>
</body>
</html>
```

4 .**Update Operation (edit.php)**

```
<?php include "db.php"; $id
= $_GET['id']; if
(isset($_POST['update'])) {
```

```
    $name = $_POST['name']; $email = $_POST['email']; mysqli_query($conn, "UPDATE users
    SET name='$name', email='$email' WHERE id=$id"); header("Location: index.php");
}

$data = mysqli_fetch_assoc(mysqli_query($conn, "SELECT * FROM users WHERE id=$id"));
?>
<form method="post">
<input type="text" name="name" value="<?= $data['name']; ?>">
<input type="email" name="email" value="<?= $data['email']; ?>">
<button name="update">Update</button>
</form>
```

5. Delete Operation (delete.php)

```
<?php
include "db.php"; $id = $_GET['id']; mysqli_query($conn,
"DELETE FROM users WHERE id=$id"); header("Location:
index.php");
?>
```

6. Session Management (session.php)

```
<?php
session_start();
$_SESSION['user'] = "Admin";
?>
```

Cookie Example

```
<?php

setcookie("username", "Student", time() + 3600); ?>
```

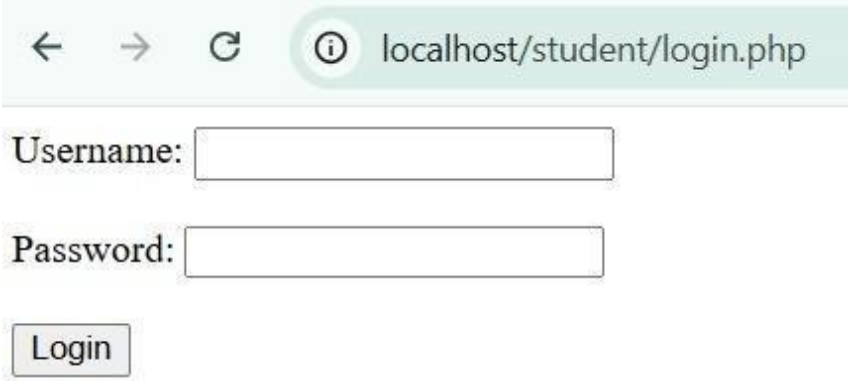
Scenario-Based Problem statement:
A small college needs a simple web system to manage student details. The administrator should be able to add new students, view the student list, update student information, and delete records using a web page connected to a MySQL database through PHP. The system must validate user input before saving data, use sessions to allow only logged-in users to access the system, and use cookies to display a welcome message to the user.

Output:

(Give output with respect to following:

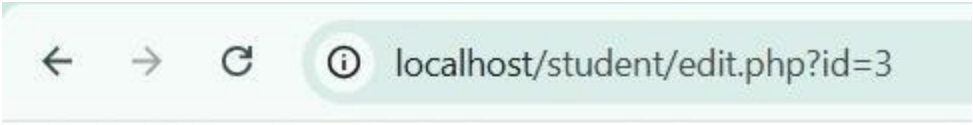
User details are inserted into the database.

- Records are displayed dynamically in tabular format.
- Users can edit and delete records.
- Sessions maintain user login state.
- Cookies store user-related information.
- All operations are performed successfully without page reload errors.)


localhost/student/login.php
Username:
Password:
Login

Student List

ID	Name	Email	Course	Action
3	John	john1234@gmail.com	BSc	Edit | Delete



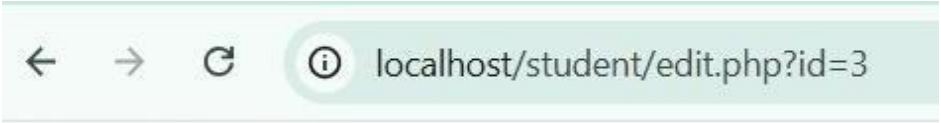
Edit Student

Name: John

Email: john1234@gmail.com

Course: BCA

Update



Updated Successfully

Edit Student

Name: John

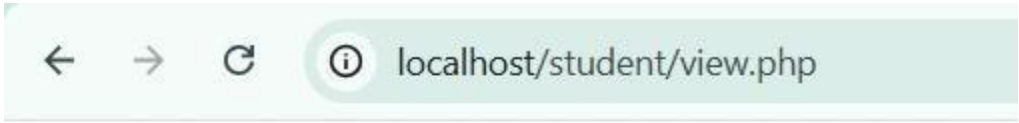
Email: john1234@gmail.com

Course: BSc

Update

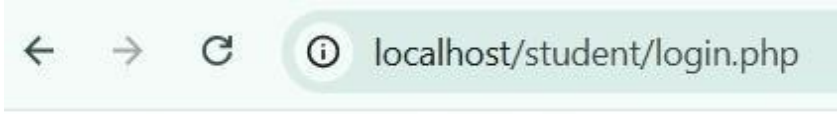
Student List

ID	Name	Email	Course	Action
3	John	john1234@gmail.com	BCA	Edit | Delete



Student List

ID	Name	Email	Course	Action



Username: admin

Password: ••••

Login

Conclusion

Thus, a dynamic database-driven web application was successfully developed using PHP and MySQL. CRUD operations were implemented with secure data handling, input validation, and session & cookie management. This experiment demonstrates real-world server-side web development concepts. **Question:**

1. What is the role of PHP in database connectivity?

PHP acts as the **translator** and **bridge** between the user's web browser and the database (like MySQL).

- **The Bridge:** PHP opens a connection to the database server.
- **The Translator:** It takes user input (like a login form) and turns it into **SQL queries** that the database understands.
- **The Messenger:** It receives the raw data back from the database, formats it into HTML/CSS, and sends it back to the user's screen.

2. Difference between session and cookie?

Sessions

- **Location:** Data is stored on the **web server**, which makes it much harder for a user to see or tamper with.
- **Security:** Highly secure. Since the actual data doesn't travel to the browser (only a "session ID" does), it is the standard choice for sensitive info like "is the user logged in?"
- **Lifespan:** Usually temporary. By default, a session expires as soon as the user closes their browser or tab.
- **Storage Capacity:** Can store a significant amount of data because it relies on server memory rather than the user's hard drive.

Cookies

- **Location:** Data is stored directly on the **user's computer** within their browser files.
- **Security:** Less secure. Users can view, edit, or even manually delete their cookies, so you should never store passwords or credit card info there.

- **Lifespan:** Can be long-lasting. You can set a cookie to expire in an hour, a month, or even years in the future (this is how "Remember Me" checkboxes work).
- **Storage Capacity:** Very limited. Most browsers only allow a maximum of **4KB** per cookie.

3. How are passwords secured in PHP?

You should **never** store passwords as plain text. If a hacker gains access to your database, they would see every user's password.

Modern PHP uses a process called **Hashing**. Unlike encryption, hashing is a one-way street; you cannot "de-hash" a password to see the original string.

- **password_hash()**: This function takes a password and turns it into a long, random string of characters using the **bcrypt** algorithm. It also automatically adds a "salt" (random data) to protect against specialized attacks.
- **password_verify()**: When a user logs in, PHP takes the password they typed, hashes it, and compares that new hash to the one stored in the database.